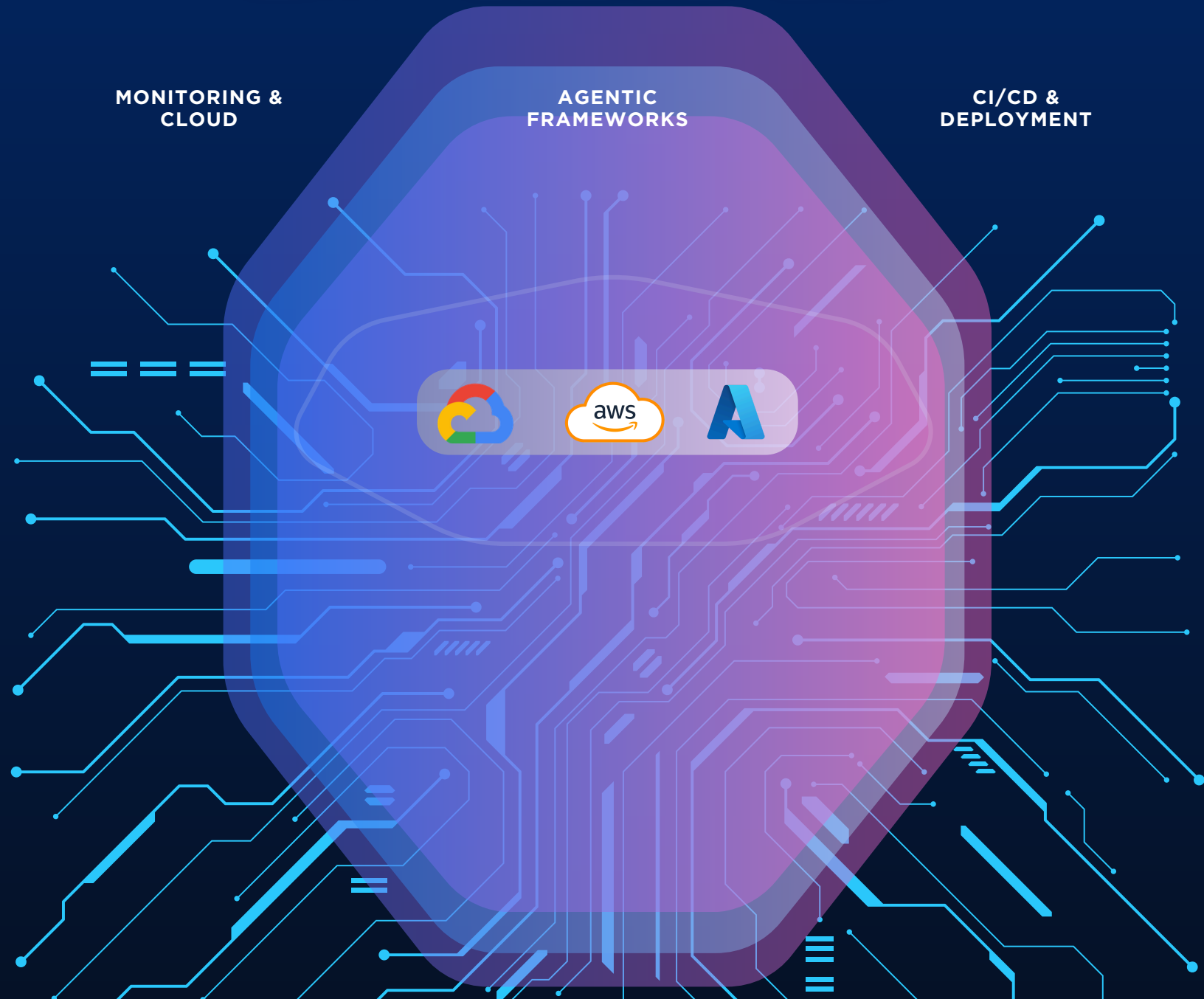
KRISH NAIK ACADEMY

AGENTIC & AGENT OPS

AI

WITH CLOUD 3.0



This course is designed for aspiring AI developers, machine learning engineers, data scientists, and software architects who want to master the 2026 agentic ecosystem. This course is suitable for those who have the **fundamentals of machine learning, deep learning & NLP as prerequisites,** as well as experienced engineers looking to transition into production-grade multi-agent systems.
This curriculum provides the high-density training required to build, deploy, and monitor autonomous AI. It is ideal for professionals aiming to lead enterprise AI initiatives using the latest protocols like MCP and A2A across multi-cloud (AWS, GCP, Azure) and local environments.

Learning Objectives

✓ Master Agentic Frameworks: Build and orchestrate intelligent systems using LangGraph, OpenAI Agents SDK, Google ADK, AWS Strands, and CrewAI.

✓ Implement Open Protocols: Deep-dive into MCP (Model Context Protocol) and A2A (Agent-to-Agent) standards for vendor-neutral tool and agent communication.

✓ Build Advanced RAG Systems: Develop production-ready Adaptive RAG, Self-RAG, and GraphRAG pipelines for complex, multi-source knowledge retrieval.

✓ Engineer Context & Reasoning: Master Context Engineering and utilize reasoning models (o3, o4-mini, Gemini Thinking) for high-reliability autonomous loops.

✓ Secure & Evaluate Agents: Implement enterprise-grade guardrails, security protocols, and trajectory-based evaluation to ensure safe and predictable autonomous actions.

✓ Apply AgentOps Lifecycle: Drive full-stack observability, tracing, and cost-monitoring using AgentOps, LangSmith, Opik, and Langfuse.

✓ Deploy Multi-Cloud Projects: Architect and ship four end-to-end portfolio projects across AWS, GCP, Azure, and Local stacks with full CI/CD integration.

Course Information

Prerequisites

No prior knowledge in Agentic AI is required, as this is a standalone path designed for those transitioning into the space. **This course is suitable for those who have the fundamentals of machine learning, deep learning & NLP as prerequisites,** While the curriculum progresses into advanced topics like cloud-native orchestration and 2026 agent protocols, the structured modules ensure that both transitioning professionals and experienced developers can accelerate their learning effectively to master production-grade autonomous systems.

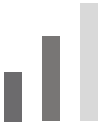
Structured as a comprehensive 5-month intensive (~6 hours/week), this course follows a "Modern & Direct Path" from foundational logic to enterprise-scale deployment. Across 26 high-density modules, students engage in a rigorous blend of recorded labs, hands-on coding exercises, and the construction of four portfolio-ready cloud projects. By bridging the gap between framework theory and production reality, the curriculum ensures absolute mastery over the full agentic lifecycle—including security, observability, and cross-cloud orchestration.

Estimated Time



5 months 6hrs/week*

Required Skill Level

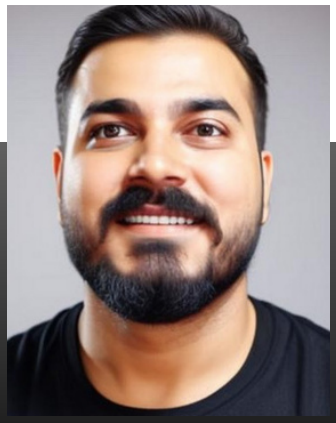


Intermediate

Krish Naik Academy Team



Mayank Aggrawal
Senior ML Engineer
Linkedin



Krish Naik
Chief AI Engineer
Linkedin



Sourangshu Pal
Senior Data Scientist
Linkedin



Monal Singh
Data Scientist
Linkedin



Sunny Savita
GenAI Engineer
Linkedin



Darius B.
Head of Product
Linkedin

Module 1

Introduction to Agentic AI

In this module, we establish the complete conceptual foundation for the course. You will understand what agents truly are, how Agentic AI differs from traditional AI and Generative AI, and survey the full landscape of frameworks and design patterns that define the field in 2025–2026. A critical focus is placed on the shift from single-agent to multi-agent systems, the emergence of the two foundational protocols (MCP and A2A), and the architectural patterns — ReAct, Supervisor, Swarm, and Pipeline — that underpin every framework in this course. By the end you will be able to articulate agent design decisions and choose the right framework for any real-world use case.

Topics	
What Are Agents?	Definition of an AI agent; Perception, Memory, Reasoning, Action loop; Agents vs. LLMs vs. Chatbots; The agentic spectrum — from simple tool-calling to fully autonomous agents
Agentic AI vs. Generative AI vs. AI Agents	Key distinctions; when to use each; why the field has shifted toward agentic systems; 2025–2026 adoption landscape
Multi-Agent Systems	What are Multi-Agents?; Benefits of specialised agents over monolithic super-agents; Orchestration patterns: Supervisor, Pipeline, Swarm, Mesh, Hub-and-Spoke; When to use each pattern
Agent Design Patterns	ReAct (Reasoning + Acting); Chain-of-Thought in agents; Plan-andExecute; Reflection and self-critique; Tool-use patterns; Memory patterns (short-term, long-term, episodic, semantic)

Topics	
The Agentic Protocol Layer	Why protocols matter: MCP for agent-to-tool, A2A for agent-to-agent; How MCP and A2A complement each other; The standardisation wave: Linux Foundation, Anthropic, Google, IBM contributions
Framework Landscape	Survey of LangGraph, OpenAI Agents SDK, Google ADK, AWS Strands, CrewAI, LlamaIndex, Claude Code, AutoGen, n8n, LangFlow; Decision framework: when to use which framework
Reasoning Models in Agents	What are reasoning models? (o3, o4-mini, DeepSeek R1, Gemini Thinking); Chain-of-thought vs. standard models; When to use reasoning models in agentic pipelines; Cost and latency tradeoffs

Module 2

Pydantic — Data Validation & Structured Outputs

Pydantic is the backbone of almost every modern agentic framework for structured data validation, tool schemas, and typed agent outputs. This module ensures you are fluent in Pydantic models before building agents that rely on typed inputs, structured tool results, and validated LLM outputs. It also introduces Logfire — Pydantic's official observability tool — which integrates into agent pipelines.

Topics	
Getting Started with Pydantic	Installation; BaseModel fundamentals; Fields, types, and default values; Validators (field_validator, model_validator); Required vs. optional fields
Core Validation & Data Handling	JSON parsing and serialisation; Nested models; Type coercion and strict mode; Dataclasses with Pydantic; Custom types and annotated fields
Advanced Validation Features	Validation decorators; Computed fields; Model Config; Settings management with pydantic-settings; Environment variable validation
Pydantic in Agentic Pipelines	Structured LLM outputs with Pydantic; Tool schemas with Pydantic models; Response validation for agent outputs; Using Pydantic with OpenAI, LangChain, and Instructor library
Monitoring with Logfire	Pydantic Logfire setup; Integrating Logfire into agentic pipelines; Tracing Pydantic validation in agent workflows; Observability for structured outputs

Module 3

LangChain & LCEL

LangChain remains the most widely-used LLM application framework and the entry point for most agentic projects. This module focuses on the production-relevant components — data ingestion, embeddings, vector stores, and LangChain Expression Language (LCEL) — that serve as building blocks for LangGraph, which follows in Module 5.

Topics	
Core Components	LangChain architecture (chains, agents, tools, memory); Introduction to basic modules; LangChain 1.0 and the create_agent abstraction
Data Ingestion	Document loaders (PDF, web, CSV, Notion, GitHub); Text splitting techniques: Recursive Character Text Splitter, Character Text Splitter, HTML Header Text Splitter, Recursive JSON Splitter; Semantic chunking
Embeddings & Vector Stores	OpenAI Embeddings; Ollama Embeddings; Hugging Face Embeddings (sentence-transformers); Vector stores: FAISS, ChromaDB, Qdrant; VectorStore retriever patterns; Hybrid search (semantic + BM25 keyword)
LCEL — LangChain Expression Language	LCEL syntax and composition patterns; Building LLM, Prompt, and Structured Output chains; Runnable interface, RunnableParallel, RunnablePassthrough; Open-source models with Groq API; Streaming with LCEL
LCEL — LangChain Expression Language	LangServe architecture and setup; Deploying LCEL runnables as REST APIs; Creating and managing custom endpoints; Playground interface; Production API patterns

Module 4

LangGraph — Stateful Agent Workflows

LangGraph is the production-grade framework for complex, stateful agent workflows. Used in production at Klarna, Replit, Uber, and LinkedIn, it provides fine-grained control over agent flow, built-in state persistence with checkpointing, and the best observability story in the ecosystem. This module covers the full LangGraph stack — from simple graphs to multi-agent deployments, state management, and human-in-the-loop patterns.

Topics	
Core Graph Concepts	Nodes, edges, and state; Directed vs. conditional edges; StateGraph API; LangGraph Studio for visual debugging; Simple graph construction; Router patterns
Agents & Memory	Single agent node; Agent with short-term memory; State schema design; State reducers; Multiple schemas; MessageGraph; Checkpointing with SQLite and Postgres backends
Message Handling	Message types (HumanMessage, AIMessage, ToolMessage); Trim and filter messages; Token budgeting strategies; Context window management in long-running agents
Human-inthe-Loop	Streaming intermediate outputs; Breakpoints and approval gates; Editing state mid-execution; Dynamic breakpoints; Time travel (rewind and resume from checkpoint); Human feedback integration

Topics	
LangGraph Deployment	Deployment concepts; LangGraph Platform (formerly LangSmith Deployment); Creating and connecting to deployments; LangGraph Cloud vs. self-hosted; Horizontal scaling and task queues; Connecting to external LangGraph deployments via API

Module 5

MCP — Model Context Protocol

MCP is Anthropic's open standard for connecting AI agents to tools, databases, and APIs in a vendor-neutral, plug-and-play way. Since its adoption boom in spring 2025 — now contributed to the Linux Foundation — MCP has become the de-facto "USB standard for AI tools." This module gives you deep practical knowledge of MCP: architecture, building custom servers, and integrating MCP into every major framework covered in this course.

Topics	
MCP Fundamentals	What is MCP and why it matters; MCP architecture: Clients, Servers, Transport layers (stdio, HTTP/SSE); Resources, Tools, and Prompts in MCP; How MCP replaces custom one-off integrations
Building MCP Servers	MCP Python SDK setup; Creating a custom MCP server from scratch; Exposing functions as MCP tools; Exposing data sources as MCP resources; Testing and debugging MCP servers locally
Community MCP Ecosystem	Official and community MCP servers (GitHub, Slack, Google Drive, Notion, Postgres, Brave Search, etc.); MCP server discovery; Composing multiple MCP servers; MCP security and authentication patterns
MCP with Agent Frameworks	MCP in LangChain and LangGraph; MCP in OpenAI Agents SDK (native support); MCP in Google ADK; MCP in AWS Strands (native support); Claude Code as an MCP server; MCP with Claude Desktop
MCP in Production	MCP server authentication (OAuth, API keys); Versioning MCP servers; Rate limiting and error handling; Running MCP servers in Docker; Deploying MCP servers to cloud; MCP governance best practices

Module 6

A2A — Agent-to-Agent Protocol

The Agent-to-Agent (A2A) Protocol — launched by Google in April 2025, now a Linux Foundation standard backed by 50+ companies including Salesforce, Atlassian, SAP, and Workday — is the "HTTP for agents." While MCP connects agents to tools, A2A connects agents to each other across frameworks and vendors. This module gives you a practical understanding of A2A that is becoming essential for enterprise multi-agent systems in 2026.

Topics	
A2A Fundamentals	What is A2A and the problem it solves; MCP vs. A2A — complementary roles (tools vs. agents); A2A architecture: Client agents, Remote agents, Agent Cards, Task lifecycle; Five design principles: agentic capabilities, open standards, security, long-running tasks, modality agnostic
A2A Core Components	Agent Cards (capability discovery via /.well-known/agent.json); Task management and lifecycle states (pending, working, completed); Message exchange and streaming; Push notifications for async tasks; A2A Python SDK
Building A2A-Compliant Agents	Wrapping an existing LangGraph agent as an A2A server; Wrapping a CrewAI agent as an A2A server; Building an A2A client to communicate with remote agents; Sequential and hierarchical A2A workflows; Crossframework A2A: ADK client ↔ CrewAI server ↔ LangGraph server
A2A in Practice	Real-world use case walkthroughs (hiring, e-commerce, healthcare); A2A with Google ADK integration; A2A deployment on Google Cloud and other platforms; Security: authentication and secure agent communication; A2A vs. OpenAI handoffs — when to use each

Module 7

RAG & Agentic RAG

Retrieval-Augmented Generation is the backbone of knowledge-grounded agents. This module progresses methodically from RAG fundamentals to fully agentic retrieval systems with selfcorrection and adaptive routing — techniques directly needed for all four cloud projects in this course.

Topics	
RAG Foundations	RAG architecture: indexing, retrieval, augmentation, generation; Chunking strategies (fixed-size, semantic, hierarchical, parent-child); Embedding models (OpenAI, Cohere, BGE, Ollama local); Vector database setup and indexing
Advanced Retrieval	Hybrid search (semantic + BM25 keyword fusion); Query rewriting and expansion; Hypothetical Document Embeddings (HyDE); Re-ranking with cross-encoders (Cohere Rerank, BGE-reranker); Metadata filtering; Contextual compression; Multi-query retrieval
RAG Evaluation	RAGAS metrics (context precision, context recall, faithfulness, answer relevancy); DeepEval for RAG evaluation; Hallucination detection; Evaluation datasets and ground truth creation; Iterative RAG improvement loops
Adaptive RAG	What is Adaptive RAG and when to use it; Implementing Adaptive RAG with LangGraph; Adaptive RAG with Cohere; Running Adaptive RAG locally; Query complexity routing: simple → direct LLM, complex → retrieval

Topics	
Agentic RAG Variants	Agentic RAG (agent decides when to retrieve); C-RAG (Corrective RAG — self-correction with web search fallback); C-RAG local implementation; SelfRAG (self-reflection and self-grading); Self-RAG local; Self-RAG with VectorDB
GraphRAG	Introduction to Knowledge Graphs for RAG; Microsoft GraphRAG architecture; LightRAG as a lightweight alternative; When GraphRAG outperforms standard vector RAG; Building a simple GraphRAG pipeline

Module 8

Context Engineering

Context engineering is the evolution beyond prompt engineering — the discipline of systematically designing, managing, and optimising the information that flows into an agent's context window at each step. As agents grow in complexity, context engineering becomes the primary lever for performance, cost, and reliability. This is an emerging but rapidly critical skill for production agentic systems in 2026.

Topics	
From Prompt Engineering to Context Engineering	Why prompt engineering alone is insufficient for agentic systems; What context engineering covers: system prompts, tool outputs, memory, retrieved documents, conversation history; The context window as a resource to manage
Structured System Prompts	Anatomy of an effective agent system prompt; Role, goal, background, instructions, constraints, output format; Dynamic system prompt generation; Versioning and testing system prompts; CLAUDE.md and AGENTS.md patterns
Memory Systems for Agents	Short-term memory (in-context); Long-term memory (external store); Episodic memory (past experience retrieval); Semantic memory (knowledge retrieval); Procedural memory (learned patterns); Implementing each type in LangGraph and LlamaIndex
Context Window Management	Token budgeting strategies; Compressing long conversation histories; Selective memory loading; Rolling windows vs. full history; LLM context summarisation; Tool output truncation strategies

Topics	
Dynamic Context Assembly	Assembling context at runtime based on agent state; Retrievalaugmented context injection; Tool results formatting for optimal LLM comprehension; Managing multi-agent shared context; Context for reasoning models (o3, o4-mini) — verbose vs. compressed
Testing & Optimising Context	Evaluating the impact of context changes on agent performance; Systematic context ablation; LLM-as-judge for context quality; A/B testing context variants; Logging and tracing context decisions with LangSmith

Module 9

OpenAI Agents SDK

Released in March 2025, the OpenAI Agents SDK is a lightweight, production-ready Python framework for multi-agent workflows. It is provider-agnostic (100+ LLMs via Chat Completions API), built on the concept of handoffs and guardrails, and used in production by Klarna, Coinbase, and Ramp. This module covers the full SDK surface including multi-agent orchestration, guardrails, sessions, tracing, MCP integration, and voice agents.

Topics	
SDK Fundamentals	Installation and configuration; Core primitives: Agent, Runner, Tools, Handoffs, Guardrails; Agent instructions and persona design; Synchronous vs. async runner; Streaming responses; Provider-agnostic LLM configuration
Tools & Tool Patterns	Function tools with @function_tool decorator; Built-in tools (web search, file search, computer use — Responses API); Connecting MCP servers as tools; Tool error handling and retries; Tool choice and forced tool calls
Multi-Agent with Handoffs	Handoff concept and implementation; Agent-as-tool pattern; Parallel agent execution; Hierarchical agent orchestration; Passing context across handoffs; Preventing circular handoffs; Real-world multi-agent build
Guardrails	Input guardrails (validating user requests); Output guardrails (validating agent responses); Tripwire pattern for safety checking; Content moderation integration; Custom business-logic guardrails; Testing guardrails

Topics	
Sessions & Tracing	Session management and conversation history; Built-in tracing and run inspection; Exporting traces to Logfire and AgentOps; Trace visualisation for debugging; Performance analysis from traces
Realtime Voice Agents	gpt-realtime-1.5 for voice agents; Automatic interruption detection; Context management in voice; Multimodal agent patterns; Building a voice-enabled agent demo

Module 10

CrewAI — Multi-Agent Collaboration Platform

CrewAI provides a role-based multi-agent framework ideal for collaborative, sequential, and hierarchical AI workflows. This module goes beyond basics to cover production customisation, CrewAI Flows (event-driven workflows), monitoring with Opik, and integration with LangChain tools.

Topics	
Core Architecture	Agents, Crews, Tasks, and Processes; Role-based agent design with Backstories, Goals, and Specialisations; Sequential vs. hierarchical process; Task assignment and delegation; Output types and structured outputs
CrewAI Flows	Event-driven workflows with Flows; @start , @listen , @router decorators; Combining Crews and Flows; State management in Flows; Conditional and parallel execution paths
Tools & Advanced Features	Built-in tools (web search, RAG, code interpreter); Creating custom CrewAI tools; LangChain tool integration; Memory types in CrewAI (short-term, long-term, entity, contextual); Guardrails and output validation; Human-inthe-loop with CrewAI
Managing Complex Crews	Managing data across agents; Role-playing patterns; Focus and constraints; Multi-crew coordination; Error handling and crew recovery; Performance optimisation

Topics	
Integration & Monitoring	LangChain and LangGraph interoperability; Monitoring CrewAI with Opik (session tracing, cost, performance); LangSmith integration; CrewAI AMP Enterprise overview; Testing crew workflows with pytest

Module 11

Google ADK — Agent Development Kit

Google's open-source Agent Development Kit — the same framework powering Google Agentspace and the Customer Engagement Suite — is the production agent framework for the GCP ecosystem. Fully model-agnostic (Gemini, Claude, GPT, Mistral via LiteLLM), it natively supports MCP, A2A, and deployment on Vertex AI Agent Engine. It is the framework anchor for the GCP project in this course.

Topics	
ADK Architecture	Core design philosophy; Agent types: LlmAgent, SequentialAgent, ParallelAgent, LoopAgent, BaseAgent; Hierarchical multi-agent composition; Description-driven delegation; Agent-as-tool patterns
Tools & MCP	Pre-built tools (Google Search, Code Execution, Computer Use); Custom function tools; MCP tool integration (native ADK support); Third-party integrations (LangChain, LlamaIndex tools); GCP service tools (BigQuery, Cloud Storage, Pub/Sub)
Model Flexibility	Gemini 2.5 Flash and Pro; LiteLLM integration for Claude, GPT, Mistral, Meta; Vertex AI Model Garden; Configuring model parameters per agent; Switching models without changing agent logic
A2A with ADK	Making ADK agents A2A-compliant (A2A server); Calling remote A2A agents from ADK (A2A client); Cross-framework A2A workflows (ADK ↔ LangGraph ↔ CrewAI); A2A deployment on Vertex AI

Topics	
Evaluation & Deployment	Built-in ADK evaluation tools; AgentEvaluator with test datasets; ADK CLI and web UI; Cloud Run deployment; Vertex AI Agent Engine (managed runtime); Scaling and session management
ADK Observability	OpenTelemetry tracing built-in; Langfuse integration; AgentOps with ADK; Cloud Monitoring and logging; Debugging with ADK web UI

Module 12

AWS Strands Agents SDK

AWS Strands (released May 2025, GA July 2025) is AWS's model-driven, lightweight agent framework. Unlike LangGraph's explicit graph definition, Strands lets the foundation model drive orchestration — define a system prompt and tools, and the LLM autonomously chains reasoning steps. It natively supports MCP, swarm patterns, and deep AWS service integration. Strands is the framework anchor for the AWS project in this course.

Topics	
Strands Architecture	Model-driven philosophy vs. code-driven (LangGraph); Agentic loop: Reason → Tool call → Observe → Repeat; System prompt as primary steering mechanism; ReAct pattern under the hood; When to choose Strands vs. LangGraph
Agents & Tools	Defining agents and tools in Strands; Python functions as tools; Built-in tools; Tool error handling; Streaming agent responses; Async agent execution
MCP Native Integration	Connecting MCP servers to Strands agents; Using community MCP servers (AWS tools, databases); Building custom MCP servers for Strands; MCP + Strands in production
Multi-Agent Patterns	Swarm (emergent coordination — no central orchestrator); Graph (deterministic routing with explicit edges); Workflow (sequential execution); Agents as tools (hierarchical delegation); State management across patterns

Topics	
AWS & Bedrock Integration	Amazon Bedrock foundation models (Claude, Nova, Titan, Llama); Bedrock Knowledge Bases (managed RAG); Session management with S3 and DynamoDB backends; Lambda, Step Functions, and ECS connectivity; AWS AgentCore Runtime (managed deployment, 8-hour async workloads)
Deployment & Observability	Deploying Strands agents on ECS Fargate; Strands on AWS Lambda; AgentCore Runtime for production; CloudWatch integration; AgentOps with Strands; Cost tracking for Bedrock foundation models

Module 13

Azure AI Foundry & Semantic Kernel

Microsoft's Semantic Kernel is the enterprise-grade SDK for building AI agents on Azure, and Azure AI Foundry is the unified platform for deploying and managing them. Combined, they represent the production environment for enterprise-scale agents — with deep Microsoft 365, Teams, and enterprise ecosystem integrations. This module is the framework anchor for the Azure project.

Topics	
Semantic Kernel Fundamentals	SK architecture: Kernel, Plugins, Functions, Memory, Planner; Connecting Azure OpenAI and other LLMs; Prompt functions and native functions; Kernel invocation patterns; Settings and dependency injection
Agent Patterns in SK	Single agent with SK; Agent Group Chat (multi-agent collaboration); Process Framework for agentic workflows; Sequential and hierarchical agent patterns; Handoffs and agent orchestration
Azure AI Foundry	Azure AI Foundry overview; Model catalog (GPT-5, Phi-4, Llama, Mistral); Project setup and management; AI Foundry Agent Service (managed agent runtime); 1000+ curated tools and MCP support

Topics	
MCP & A2A in SK + Foundry	Semantic Kernel MCP plugin support; A2A compliance for SK agents; Connecting agents to enterprise data sources; Azure AI Agent Service with MCP; Cross-platform agent communication
Azure Deployment	Azure Container Apps deployment; Managed identities and Key Vault; Environment-based configuration; AI Gateway for runtime guardrails; Entra Agent Identity for agent security
Monitoring on Azure	Azure Monitor and Application Insights; Agent-specific telemetry; Custom dashboards for agent performance; Alert rules and incident response; AgentOps integration with Azure deployments

Module 14

LlamaIndex — Data-Centric Agents

LlamaIndex excels as the "data brain" behind intelligent agents — providing semantic indexing, persistent memory, agentic retrieval workflows, and a unique event-driven workflow system. This module focuses on its agentic capabilities beyond basic RAG, including LlamaIndex Workflows for complex multi-step agent orchestration.

Topics	
LlamaIndex Core	Documents, Nodes, and Indexes (Vector, Summary, Knowledge Graph); Query engines and chat engines; Ingestion pipeline with transformations; LlamaIndex Storage: DocStore, IndexStore, VectorStore
Agentic Workflows	LlamaIndex Workflows: event-driven, step-based orchestration; @step decorator pattern; Workflow state and context management; Streaming workflows; Combining multiple steps into complex pipelines
Agent Tools & Memory	QueryEngineTool and FunctionTool; OpenAI function-calling agent; ReAct agent in LlamaIndex; Long-term memory with LlamaIndex Memory; Multidocument agents; Agent with RAG as a tool

Topics	
LlamaIndex + MCP	Integrating MCP tools into LlamaIndex agents; Using LlamaIndex as a tool within other frameworks; LlamaIndex + LangGraph hybrid architectures; LlamaIndex + OpenAI Agents SDK
Advanced Indexing	Property Graph Index (LlamaIndex GraphRAG); Auto-merging retriever; Sentence window retrieval; Recursive retrieval for hierarchical documents; Sub-question query engine

Module 15

Claude Code — Agentic Coding

Claude Code is Anthropic's command-line agentic coding tool with direct file system and terminal access. It represents a new class of coding agent — not just a code completion tool, but a fully autonomous agent that can plan multi-step software tasks, run tests, debug, and commit code. This module teaches you how to use Claude Code for real workflows, extend it with MCP, and integrate it into agentic pipelines.

Topics	
Claude Code Fundamentals	Installation and setup; CLI usage patterns; CLAUDE.md for project context; Permissions model (read/write/execute); Session management; Cost and usage tracking
Agentic Coding Workflows	Repository-scale code understanding; Automated code generation and refactoring; Test writing and running; Bug identification and fixing; Documentation generation; Multi-file code changes
MCP with Claude Code	Running Claude Code as an MCP server (exposing to other agents); Connecting MCP servers to Claude Code (tools); Building custom MCP integrations; Claude Code + LangGraph hybrid workflows

Topics	
Extended Thinking & Automation	Using extended thinking for complex architectural decisions; Autonomous multi-step tasks; AGENTS.md for AI coding guidelines; CI/CD integration with Claude Code; GitHub Actions + Claude Code
Claude Code in Multi-Agent Pipelines	Using Claude Code as a specialised coding sub-agent; Handoffs to Claude Code from orchestrator agents; Results handling and code review workflows; Combining Claude Code with OpenAI Agents SDK handoffs

Module 16

n8n — AI Workflow Automation

n8n is a powerful open-source workflow automation platform with deep AI integration. It sits in a unique position in the agentic ecosystem — between low-code and full-code — making it ideal for building production automation pipelines that connect AI agents to real business systems (CRMs, databases, messaging platforms, APIs) without writing extensive backend code.

Topics	
n8n Fundamentals	n8n architecture: workflows, nodes, connections; Interface walkthrough; Core workflow concepts (triggers, actions, branching); Data structures in n8n (items, binary data); Expressions and the Code node; Credentials management
APIs, Webhooks & Error Handling	HTTP Request node for external APIs; Webhook triggers (synchronous and asynchronous); Error workflow patterns; Retry logic and fallbacks; Debugging with execution history; Node execution modes
AI Agents in n8n	AI Agent node deep-dive; Connecting LLMs (OpenAI, Anthropic, Gemini, Ollama); Memory nodes (Window Buffer, Redis, PostgreSQL); Tool nodes in AI Agent (calculator, web search, SQL, etc.); Building RAG workflows with vector store nodes; Chat interfaces

Topics	
Automation Projects	Building an automated RAG chatbot; Blog post generation pipeline with WordPress; Automated meeting transcription and summarisation with Notion; Lead enrichment workflow; Customer support automation pipeline
Integrations & Production	WhatsApp and Telegram bot integration; Calendar and email automation; Google Sheets and Airtable integrations; Webhook-based agent triggers; n8n self-hosting with Docker; Scaling n8n with queue mode

Module 17

LangFlow — Low-Code Agent Builder

LangFlow is a low-code visual platform for building LLM-powered workflows and agent applications. It lowers the barrier to prototyping agentic systems and is increasingly used by nontechnical stakeholders to build and iterate on agent applications. This module covers setup, visual workflow design, third-party integrations, chatbot deployment, and how to bridge LangFlow with production code.

Topics	
LangFlow Setup & UI	Installation (local and cloud); Environment setup; LangFlow UI walkthrough; Key terminologies: Nodes, Chains, Prompts, Components; Navigating the component library
Building Workflows	Creating your first LangFlow application (quickstart); Understanding prebuilt vs. custom workflows; LLM integration in LangFlow; Prompt engineering in the LangFlow visual editor; Commonly used nodes (LLM, Prompt, Output Parser, Memory, Vector Store)
Third-Party Integrations	Connecting data sources: SQL, CSV, NoSQL databases; API integration for external services (REST); Vector database integration for embeddings; LangFlow with OpenAI and Hugging Face models; LangChain component integration
Building Chatbots	Automating chat workflows; Building a complete chatbot application; Memory-enabled conversations; Export and API access for LangFlow flows; Embedding LangFlow in applications

Topics	
LangFlow to Code Bridge	Exporting LangFlow workflows to Python code; Iterating between visual and code approaches; When to move from LangFlow to LangGraph; Using LangFlow as a rapid prototyping tool

Module 18

AutoGen — Conversational Multi-Agent Framework

Microsoft AutoGen (now AG2) enables multi-agent systems built as structured conversations. This module covers the current AG2 architecture (v0.4+ redesigned event-driven system), multi-agent collaboration patterns, feedback loops, and deployment with performance monitoring.

Topics	
AutoGen Framework	Overview and evolution (AutoGen → AG2 v0.4 redesign); Key concepts: Autonomy, Adaptability, Inter-Agent Communication; Installation and environment setup; Core, AgentChat, and Extensions APIs
Agent Types & System Design	AssistantAgent, UserProxyAgent, and custom agents; Group chat and nested chat patterns; Agent roles, goals, environments, and actions; Designing and developing agentic decision-making frameworks
Agent Interactions & Learning	Conversation-driven multi-agent execution; Implementing feedback loops; Handling uncertainty and constraints; Agent learning and adaptation patterns; Human-in-the-loop with UserProxy
Multi-Agent Collaboration	Multi-agent collaboration patterns; Parallel and sequential execution; Error handling across agent conversations; Integrating external tools and APIs; AutoGen + LangChain/LangGraph interop

Topics	
Deployment & Monitoring	Containerising AutoGen applications; Deployment on cloud (AWS, Azure, GCP); Monitoring agent performance; Integrating AgentOps for AG2; Testing multi-agent conversations

Module 19

Agent Security, Guardrails & Evaluation

As agents become more autonomous and take real actions, security and reliability are nonnegotiable. This module covers the emerging discipline of agent safety — from prompt injection attacks to guardrail implementation and systematic evaluation. These concepts apply across every framework in this course and are critical for production deployments.

Topics	
Agent Security Fundamentals	The agentic attack surface: prompt injection, tool misuse, data exfiltration; Least-privilege access control for agents; Secure tool execution environments; Tamper-resistant audit logging for agent actions; API security: rate limiting, authentication, authorisation; Agent identity and Entra Agent Identity patterns
Prompt Injection & Defences	Direct vs. indirect prompt injection; How malicious MCP server responses can hijack agents; Defences: input sanitisation, output validation, sandboxed tool execution; Testing agents for prompt injection resistance
Guardrails Implementation	Types of guardrails: input, output, action; NeMo Guardrails overview; Guardrails AI hub and custom validators; OpenAI Agents SDK guardrails (input/output tripwires); LangGraph interrupt and approval patterns; Human approval gates for high-risk actions

Topics	
Agent Evaluation	Why agent evaluation is harder than model evaluation; Trajectory-based evaluation (step-by-step execution quality); Task completion evaluation; LLM-as-judge for agent responses; DeepEval for agent testing; RAGAS for RAG component evaluation; Building evaluation datasets; Automated evaluation pipelines in CI/CD
Safety Patterns for Production	Defining strict action boundaries; Cryptographically signed audit logs; Emergency stop mechanisms; Multi-agent security: encrypted interagent communication; Compliance and regulatory considerations (GDPR, HIPAA); NIST AI RMF for agentic systems

Module 20

AgentOps — The Full Observability Lifecycle

AgentOps is more than a tool — it is an emerging discipline (like DevOps was for software) that covers the full lifecycle of autonomous AI agents from prototype to production. This dedicated module covers the complete AgentOps stack: tracing, debugging, cost management, evaluation, and production monitoring across all major frameworks, with deep hands-on sessions with AgentOps SDK, LangSmith, Opik, and Langfuse.

Topics	
What Is AgentOps?	AgentOps vs. MLOps vs. DevOps; Why agents need specialised observability (non-deterministic, multi-step, tool-calling); The AgentOps lifecycle: build → evaluate → monitor → optimise; Key metrics: response time, cost per run, tool call success rate, task completion rate, hallucination rate
AgentOps SDK	Setup with 2 lines of code; Session tracking and session replays; LLM call tracing and visualisation; Tool usage monitoring; Multi-agent interaction tracking; Cost tracking and budget alerts
LangSmith	LangSmith setup and configuration; Tracing LangChain and LangGraph runs; Evaluation datasets and test suites; Prompt versioning and A/B testing; Automating evaluations with LangSmith; Hub sharing
Opik by Comet	Opik setup and integration; Monitoring CrewAI applications with Opik; LLM evaluation features; Prompt versioning; Comparing model versions; Opik dashboard walkthrough

Topics	
Langfuse	Langfuse as an open-source observability platform (MIT license); Selfhosting Langfuse with Docker; LLM-as-judge evaluations; Annotation queues; Prompt experiments and playground; Integrating with LangChain, LlamaIndex, OpenAI SDK
Comparing Observability Tools	When to use AgentOps vs. LangSmith vs. Opik vs. Langfuse; Latency overhead comparison; Cost of observability at scale; Choosing the right tool for your framework; Combining tools in a unified observability stack
Production Monitoring Strategies	Defining SLOs for agent systems; Alerting on agent failures and cost overruns; Anomaly detection in agent behaviour; Continuous evaluation in production; Incident response runbooks for agent failures

Module 21

CI/CD, Deployment & Production Serving

This module brings together the complete deployment pipeline for AI agents. You will build Docker containers, CI/CD pipelines with GitHub Actions, and learn three production-grade serving patterns: FastAPI, BentoML, and cloud-native containers. This module directly prepares you for all four cloud projects.

Topics	
Containerisation	Docker fundamentals for AI agents; Writing optimised Dockerfiles (multi-stage builds, layer caching); Managing model weights and dependencies in containers; Docker Compose for local multi-service development; Container security best practices
CI/CD with GitHub Actions	GitHub Actions architecture (workflows, jobs, steps, runners); Building and testing AI agent code in CI; Building and pushing Docker images to ECR/GCR/ACR; Automated deployment workflows; Branch protection and environment approvals; Secrets management in GitHub Actions
FastAPI for Agent APIs	FastAPI project structure for agents; Async endpoint design for longrunning agents; Streaming responses (Server-Sent Events); WebSocket for real-time agent interaction; Pydantic request/response validation; API documentation, health checks, and middleware

Topics	
BentoML	BentoML architecture: Services, Runners, and Bentos; Packaging AI agents as Bentos; Serving locally and scaling; Monitoring with BentoML's built-in dashboard; Deploying to BentoCloud; GPU resource management
Cloud Deployment Patterns	AWS: ECS Fargate, App Runner, Lambda (serverless agents); GCP: Cloud Run (serverless containers), Vertex AI Agent Engine; Azure: Container Apps, Azure Functions; Choosing the right pattern based on workload (request-response, async, long-running)
Infrastructure as Code	Managing cloud resources with AWS CDK or Terraform basics; Environment variable and secret management across environments (dev/staging/prod); Blue-green deployments; Auto-scaling configuration for agent workloads

Module 22

Designing Multi-Agent Systems in Production

A dedicated deep-dive module on architecting, building, and testing production-grade multi-agent systems using LangGraph. This module consolidates all the multi-agent patterns from previous modules and applies them to a real production scenario with full observability and evaluation.

Topics	
System Architecture Design	Requirements to architecture translation; Choosing between Supervisor, Swarm, Pipeline, and Mesh patterns; State schema design for complex systems; Agent specialisation strategy; Communication protocol choice (LangGraph state, A2A, or direct handoffs)
Building Agent Nodes & Roles	Building agent nodes in LangGraph; Defining tasks, responsibilities, and scope per agent; Tool allocation per agent; Preventing overlap and conflicts between agents
Coordination Mechanisms	Supervisor agent implementation; Dynamic routing with conditional edges; Shared memory and state management; Parallel agent execution patterns; Retry and fallback strategies
Scalability & Reliability	Stateful coordination at scale; Checkpointing for long-running workflows; Fault tolerance and agent recovery; Testing multi-agent workflows: unit, integration, load testing; Chaos engineering for agent systems

Topics	
Real-World Multi-Agent Build	End-to-end build walkthrough: requirements → architecture design → implementation → testing → monitoring → deployment; Architecture diagrams with Excalidraw

Module 23

Enterprise Capstone — Multi-Cloud & Local Agentic Architectures

A high-intensity, project-driven module designed to transition from theoretical patterns to production-grade deployments. You will architect, build, and deploy four distinct agentic systems across AWS, GCP, Azure, and Local stacks, mastering framework-specific SDKs (Strands, ADK, CrewAI, LangGraph), MCP-integrated tooling, and enterprise-grade CI/CD pipelines with full observability.

Topics	
Project 1 — AWS: Multi-Agent Business Automation	Architecting for Business Intelligence: Designing a supervisor-led multi-agent pipeline using AWS Strands SDK and Bedrock; Infrastructure as Code: Deploying to ECS Fargate with S3, DynamoDB, and Secrets Manager; Production Workflows: Implementing async long-running tasks (8-hour support), custom MCP servers on Lambda, and GitHub Actions CI/CD; Enterprise Monitoring: Full session replay with AgentOps and distributed tracing via AWS X-Ray.
Project 2 — GCP: RAG + MCP Knowledge Assistant	Enterprise RAG Pipeline: Building multi-source retrieval (Google Drive, SQL, PDFs) using Vertex AI Matching Engine and Hybrid Search; ADK Development: Building agents with Google ADK and Gemini 2.5 Flash; A2A Compliance: Implementing AgentCard and A2A-compatible servers on Cloud Run; GCP Observability: Deploying Langfuse on GCP and monitoring RAG performance with RAGAS evaluation pipelines.

Topics	
Project 3 — Azure: Customer Intelligence Platform	Role-Based Collaboration: Designing a CrewAI "Crew" with specialized roles for sentiment, competitor benchmarking, and reporting; Azure Vector Stack: Integrating Azure AI Search and Cosmos DB for state persistence; Scaling & Security: Deploying to Azure Container Apps with Managed Identity and Key Vault; Telemetry & Insights: Custom telemetry with Application Insights and automated crew evaluation in CI using DeepEval.
Project 4 — Local: Fine-Tuned Agent & Privacy Stack	Domain-Specific Fine-Tuning: Instruction-tuning models using LoRA/QLoRA and exporting to GGUF for Ollama; Local Agentic Pipeline: Building a LangGraph RAG system using Chroma DB and local embedding models; Self-Hosted Orchestration: Orchestrating the full stack (Ollama, FastAPI, Streamlit, Langfuse) via Docker Compose; Zero-Cost Observability: Benchmarking fine-tuned local models vs. GPT-4o for latency and response quality.